

Memoria del Proyecto: ApexFlow

Desarrollo de un Sistema SaaS de Telemetría y Análisis para SimRacing

1. Introducción y Motivación

El automovilismo de competición es un deporte donde las milésimas de segundo marcan la diferencia entre la victoria y la derrota. En la vida real y en los simuladores de alto nivel (SimRacing), la herramienta más poderosa de un piloto no es el volante, sino los datos. Sin embargo, el acceso a software de telemetría profesional suele estar limitado a ingenieros, requiriendo conocimientos técnicos avanzados para interpretar hojas de cálculo interminables o gráficas poco intuitivas.

La motivación principal detrás de **ApexFlow** nació de una necesidad personal: democratizar el acceso a la telemetría. Mi objetivo era construir un sistema que no solo capturara los datos en tiempo real desde un simulador (Assetto Corsa), sino que los procesara y los presentara de forma visual, atractiva y comprensible a través de una arquitectura web moderna.

Este proyecto no es un simple panel de visualización; es una arquitectura completa "End-to-End" que abarca desde la extracción de datos binarios en memoria (Edge/IoT) hasta la persistencia en bases de datos y la visualización analítica mediante algoritmos de comparación matemática.

2. Arquitectura del Sistema

Para garantizar que el sistema fuera escalable y de baja latencia, decidí huir de las aplicaciones monolíticas tradicionales y diseñar una arquitectura fuertemente desacoplada. El proyecto se divide en tres capas principales:

- **Capa de Extracción (Agente Local):** Un script en Python encargado de interceptar las comunicaciones de red del simulador de carreras.
- **Capa de Procesamiento (Backend/Cloud):** Un servidor desarrollado con FastAPI que gestiona las conexiones simultáneas, emite los datos en vivo y almacena el histórico.
- **Capa de Presentación (Frontend):** Una interfaz web reactiva y asíncrona para la visualización del panel de instrumentos y el análisis comparativo de vueltas.

Esta separación permite que, en el futuro, el backend y el frontend puedan alojarse en un servidor en la nube (AWS, Render, Railway), mientras que el agente local sigue corriendo en el PC del usuario, convirtiendo a ApexFlow en un verdadero producto SaaS (Software as a Service).

3. Ingesta de Datos: El Agente IoT y la Barrera Binaria

El primer gran reto técnico fue la obtención de los datos. Los simuladores profesionales como Assetto Corsa no exportan su telemetría en formatos legibles como JSON o XML debido a las necesidades extremas de rendimiento. En su lugar, emiten bloques de memoria comprimidos en código binario (C++) a través del protocolo UDP.

Para solucionar esto, desarrollé `agent.py`. El flujo de trabajo de este agente es el siguiente:

1. **Handshake:** Abre un socket UDP y se comunica con el puerto local 9996 del simulador, enviando una petición formal de suscripción para iniciar la transmisión.
2. **Desempaquetado Binario:** Utilizando la librería nativa `struct` de Python, el agente mapea y traduce la cadena de bytes recibida. Tuve que investigar la estructura de datos `RTCarInfo` del motor del juego para extraer con precisión variables como la velocidad (float), RPM (float), y entrada de pedales (gas, brake).
3. **Transmisión:** Los datos desempaquetados se formatean y se envían a un ritmo de 60Hz a través de WebSockets hacia el servidor central.

4. Backend: Procesamiento Asíncrono y Persistencia

El corazón de la aplicación es `cloud_backend.py`. Elegí **FastAPI** por su excelente manejo del asincronismo y su rendimiento nativo.

Comunicación en Tiempo Real (WebSockets)

Para un panel de instrumentos de un coche de carreras, el protocolo HTTP tradicional (basado en peticiones y respuestas) era inviable; el *polling* constante colapsaría el servidor y generaría lag. Implementé WebSockets para mantener un túnel bidireccional abierto. El servidor recibe la telemetría del Agente y realiza un *broadcast* instantáneo a todos los clientes web conectados.

Almacenamiento Eficiente (SQLite)

Guardar cada paquete de datos recibido a 60 hercios destruiría cualquier base de datos en minutos si se hiciera inserción a inserción. Para optimizar el I/O (Input/Output), diseñé un sistema de "lotes" (batching). El backend acumula los datos en la memoria RAM y, cada cierto tiempo, realiza una única inserción masiva en una base de datos **SQLite**. Esto asegura que el rendimiento en tiempo real no se vea penalizado por la escritura en disco.

5. El Reto Matemático: Sincronización de Trazadas (Ghost Lap)

Una de las características clave que quería implementar era el "Modo Análisis": la capacidad de superponer la gráfica de una vuelta actual con una vuelta de referencia (el piloto profesional o *Ghost Lap*). Aquí me enfrenté al desafío técnico más complejo del proyecto.

Inicialmente, las librerías gráficas apilaban los datos según el índice del array temporal (es decir, el dato nº 500 de la Vuelta 1 se comparaba con el dato nº 500 de la Vuelta 2). Esto generaba un problema gravísimo: las "Outlaps" (vueltas de salida). Si el coche estaba parado en boxes durante 10 segundos, se generaban miles de puntos de datos con velocidad "0 km/h". Al superponer las gráficas, las curvas de frenada y aceleración quedaban completamente desalineadas, volviendo la herramienta inútil.

La Solución: Decidí abandonar el tiempo como medida de referencia y pivotar hacia una métrica geoespacial. Modifiqué el agente para que calculara la distancia exacta recorrida mediante la fórmula del Movimiento Rectilíneo (Distancia = Velocidad × Tiempo transcurrido entre *frames*).

En el frontend, reescribí el renderizado de la gráfica para utilizar un **eje X lineal**. Transformé las listas de datos en coordenadas cartesianas estrictas (X: Metros en pista, Y: Velocidad). Gracias a esto, no importa si el coche estuvo parado media hora en la línea de meta; el metro 450 de la pista siempre se alinearía perfectamente con el metro 450 de la vuelta de referencia.

6. Diseño UI/UX y Visualización de Alto Rendimiento

La interfaz de usuario no podía ser una simple tabla de números. Quería replicar el entorno de alta presión y tecnología de un muro de boxes de Fórmula 1.

Utilicé **Tailwind CSS** para construir una interfaz responsiva y moderna. Adopté un diseño basado en *Glassmorphism* (paneles oscuros translúcidos, desenfoques y bordes sutiles iluminados) para reducir la fatiga visual.

- **Live Dashboard:** Incluye visualizadores LCD digitales, barras de progreso verticales para la telemetría de los pedales y un gradiente de color dinámico para la barra de RPM que simula el corte de inyección.
- **Motor Gráfico:** Integré **Chart.js**, desactivando las animaciones en tiempo real y optimizando el *rendering* de los ejes para que el navegador pudiera dibujar cientos de puntos por segundo sin sufrir caídas de fotogramas (FPS).

7. Innovación: El Ingeniero de Pista Heurístico

No quería que el proyecto se quedara en una simple herramienta de pintado de gráficas; quería que aportara valor real al usuario. Por ello, desarrollé el "Coach Inteligente".

Implementé un algoritmo heurístico en el servidor que se ejecuta cuando el usuario solicita una comparativa. El sistema cruza los diccionarios de datos de ambas vueltas, buscando discrepancias matemáticas superiores a un umbral definido. Si detecta que en un metro específico de la pista la velocidad del usuario es 20 km/h inferior a la de la vuelta de referencia, el algoritmo evalúa el contexto:

- Si la velocidad global es baja, infiere que es el vértice de una curva e imprime: *"Pierdes velocidad en el paso por curva. Intenta dejar correr más el coche"*.
- Si la diferencia se da en una zona de alta velocidad descendente, infiere que es un error de frenada e imprime: *"Frenas demasiado pronto frente a la referencia"*.

Esta capa de procesamiento de lenguaje natural añade un factor diferencial enorme al producto, acercándolo a soluciones de nivel empresarial.

8. Conclusiones

El desarrollo de ApexFlow ha supuesto un viaje exhaustivo por múltiples disciplinas de la ingeniería de software. Ha requerido comprender protocolos de red de bajo nivel (UDP), dominar la asincronía en Python, diseñar esquemas de bases de datos para series temporales de alta frecuencia, resolver problemas matemáticos de alineación de datos y construir una interfaz de usuario atractiva y reactiva.